

1 GeoRakshak: PostgreSQL/PostGIS to XGBoost Risk Prediction Architecture

Technical Design and Implementation Document

Project: GeoRakshak — AI-Based Landslide Early Detection and Management System

Document version: 1.0

Prepared for: Smart India Hackathon project development and GitHub implementation

Prepared by: GeoRakshak Project Team

Date: 10 September 2026

1.1 Abstract

This document defines how GeoRakshak can use PostgreSQL and PostGIS as the system of record and XGBoost as a supervised machine-learning model for landslide-risk estimation. The recommended approach converts heterogeneous database records into a structured, versioned feature vector for each hazard zone and observation time. Features may include terrain, geology, soil, rainfall windows, forecast rainfall, soil moisture, tilt, pore-water pressure, historical landslide activity, exposure, and data quality.

The document deliberately distinguishes between a structured feature vector and an embedding vector. XGBoost should primarily use structured physical and geospatial features. PostgreSQL extensions such as pgvector may be used later for optional text, image, and similarity-search capabilities, but embeddings should not replace the core physical feature representation.

The proposed implementation begins with a transparent rainfall-threshold and susceptibility baseline. XGBoost is introduced after GeoRakshak has accumulated reliable, time-stamped, geolocated, and human-verified landslide records. The system must use time-based and spatial validation, address rare-event imbalance, prevent data leakage, expose prediction evidence, and retain complete model and alert audit trails.

Safety statement: XGBoost produces a risk estimate. It does not guarantee that a landslide will occur or fail to occur. Public warnings and evacuation instructions must remain governed by an approved human and institutional operating procedure.

1.2 1. Purpose and scope

The purpose of this document is to provide an implementable design for extracting GeoRakshak data from PostgreSQL/PostGIS, generating model features, training and serving an XGBoost model, and converting the output into operational risk states.

The document covers the following areas:

- structured feature-vector design;
- PostgreSQL and PostGIS data organization;
- feature engineering for rainfall, terrain, sensors, and exposure;
- target-label construction;
- XGBoost training and validation;
- risk-score calibration and warning states;

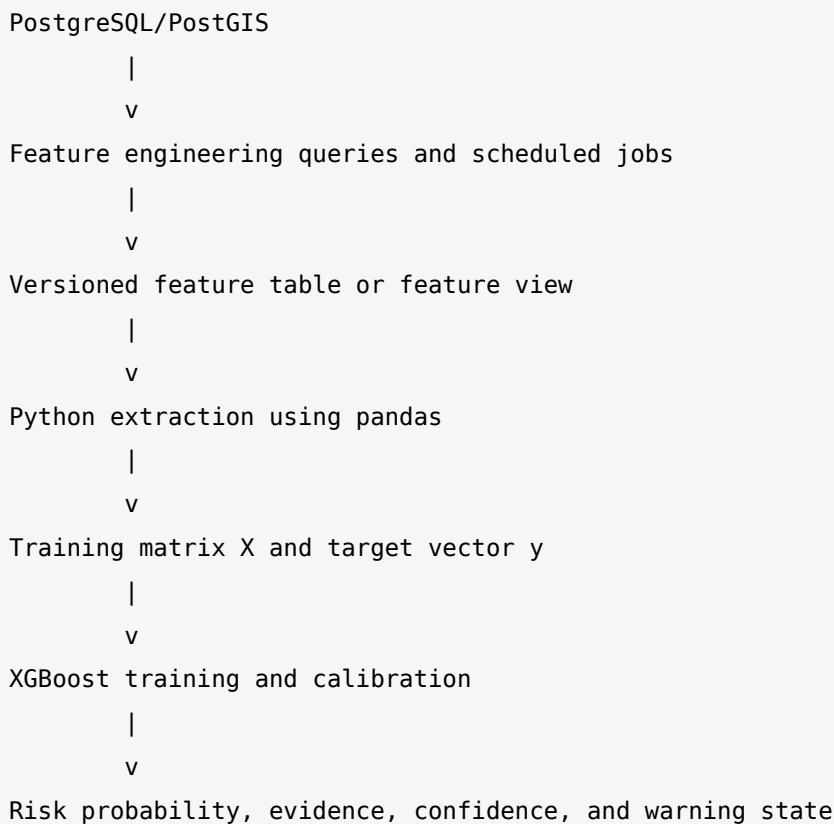
- explainability, data quality, and model governance;
- optional use of pgvector for unstructured data;
- implementation phases and acceptance criteria.

This document does not define a final geotechnical threshold for a specific district. Thresholds must be calibrated using local data and approved by qualified domain experts and responsible authorities.

1.3 2. Recommended technical position

Yes, GeoRakshak can use XGBoost by converting PostgreSQL data into feature vectors. This is an appropriate approach because XGBoost performs well on tabular data containing numerical, categorical, missingness, and engineered time-window features.

The recommended data flow is:



The database should remain the source of truth. The model should consume a reproducible feature snapshot and write its result back with the model version, feature timestamp, evidence, and expiry time.

1.4 3. Feature vectors versus embeddings

1.4.1 3.1 Structured feature vector

A structured feature vector is a fixed-length row containing physically meaningful variables. A simplified GeoRakshak vector may be represented as:

```
[
  slope_degrees,
  elevation_m,
```

```

rainfall_1h_mm,
rainfall_6h_mm,
rainfall_24h_mm,
rainfall_72h_mm,
soil_moisture,
tilt_change_1h,
pore_pressure_change,
distance_to_road_m,
historical_event_count,
exposed_population,
sensor_health_score
]

```

This is the preferred input representation for XGBoost because each variable has a clear operational interpretation.

1.4.2 3.2 Embedding vector

An embedding is a numerical representation generated from unstructured content such as text, images, inspection notes, or satellite-image patches. An embedding may help search for similar reports or triage photographs, but it is not a substitute for rainfall, slope, moisture, or movement data.

The project may add PostgreSQL pgvector later for the following functions:

- similarity search across historical community reports;
- retrieval of similar verified landslide photographs;
- search over multilingual field-inspection notes;
- storage of embeddings for satellite-image tiles.

The primary risk model should remain based on structured physical and geospatial variables.

1.5 4. Data model

1.5.1 4.1 Training feature table

The following table is a practical starting point for a pilot deployment:

```

CREATE TABLE landslide_training_features (
  id BIGSERIAL PRIMARY KEY,
  zone_id BIGINT NOT NULL,
  observation_time TIMESTAMPTZ NOT NULL,

  slope_degrees DOUBLE PRECISION,
  elevation_m DOUBLE PRECISION,
  soil_moisture DOUBLE PRECISION,

  rainfall_1h_mm DOUBLE PRECISION,
  rainfall_6h_mm DOUBLE PRECISION,
  rainfall_24h_mm DOUBLE PRECISION,

```

```

rainfall_72h_mm DOUBLE PRECISION,

rainfall_forecast_6h_mm DOUBLE PRECISION,
rainfall_forecast_24h_mm DOUBLE PRECISION,

tilt_change_1h DOUBLE PRECISION,
tilt_change_6h DOUBLE PRECISION,
pore_pressure_change DOUBLE PRECISION,

road_cut_distance_m DOUBLE PRECISION,
drainage_distance_m DOUBLE PRECISION,
historical_event_count INTEGER,

exposed_population INTEGER,
sensor_health_score DOUBLE PRECISION,

event_in_next_6h BOOLEAN,
label_confidence DOUBLE PRECISION,

feature_version TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

The event_in_next_6h field is the target label. It should be true only when a verified landslide event intersects the zone in the following six hours. If monitoring or reporting coverage is inadequate, the period should be marked unknown rather than treated as a safe negative.

1.5.2 4.2 Core operational entities

Entity	Purpose
hazard_zone	Stores a susceptibility polygon or grid cell and its source version.
sensor	Stores sensor type, location, calibration, ownership, and health.
observation	Stores time-stamped sensor values and quality flags.
weather_observation	Stores rainfall, forecast, station, source, and timestamp.
landslide_event	Stores verified or provisional event geometry, timing, source, and confidence.

Entity	Purpose
community_report	Stores resident or field-worker reports and media references.
risk_assessment	Stores score, warning state, evidence, model version, and expiry.
alert	Stores approved public messages and authority decisions.
delivery	Stores SMS, IVR, siren, push, or public-address delivery outcomes.
audit_log	Stores immutable or append-only operational history.

1.5.3 4.3 PostGIS usage

PostGIS should be enabled for spatial operations:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

PostGIS can calculate distance to roads and drainage, determine which buildings or schools intersect a hazard polygon, find the nearest sensor, aggregate population exposure, and assign weather observations to hazard zones.

Example distance feature query:

```
SELECT
  z.id AS zone_id,
  MIN(ST_Distance(
    z.geometry::geography,
    r.geometry::geography
  )) AS distance_to_road_m
FROM hazard_zones z
JOIN roads r
  ON ST_DWithin(
    z.geometry::geography,
    r.geometry::geography,
    5000
  )
GROUP BY z.id;
```

1.6 5. Feature engineering

1.6.1 5.1 Rainfall features

Use several rainfall windows rather than a single daily total. Candidate features include rolling rainfall over 1, 3, 6, 12, 24, 48, and 72 hours, maximum short-duration intensity, deviation from seasonal baseline, antecedent rainfall index, and forecast rainfall over the next 6 to 24 hours.

The India Meteorological Department publishes APIs covering current weather, nowcast, AWS/ARG data, district warnings, district rainfall, and forecast products [1](#). GeoRakshak should retain each source timestamp, source identifier, retrieval time, and raw reference.

Example feature query:

```
SELECT
  zone_id,
  observation_time,
  SUM(rainfall_mm) FILTER (
    WHERE observed_at >= observation_time - INTERVAL '1 hour'
  ) AS rainfall_1h_mm,
  SUM(rainfall_mm) FILTER (
    WHERE observed_at >= observation_time - INTERVAL '6 hours'
  ) AS rainfall_6h_mm,
  SUM(rainfall_mm) FILTER (
    WHERE observed_at >= observation_time - INTERVAL '24 hours'
  ) AS rainfall_24h_mm,
  SUM(rainfall_mm) FILTER (
    WHERE observed_at >= observation_time - INTERVAL '72 hours'
  ) AS rainfall_72h_mm
FROM weather_observations
GROUP BY zone_id, observation_time;
```

1.6.2 5.2 Terrain and susceptibility features

Static features should include slope angle, elevation, aspect, curvature, relative relief, drainage proximity, topographic wetness proxy, road-cut proximity, geology, soil class, land cover, vegetation loss, faults or lineaments where available, and historical event density.

The North Eastern Regional Node for Disaster Risk Reduction describes landslide inventories, susceptibility mapping, and the use of geological, geomorphological, land-use, drainage, soil, and topographic parameters [2](#). These features should be versioned with their source resolution and preprocessing method.

1.6.3 5.3 Sensor features

Use both the current value and the change over time. Candidate features include:

- soil-moisture level and rate of change;
- tilt change over 1, 6, and 24 hours;
- displacement velocity and acceleration;
- pore-water pressure change;

- vibration anomaly;
- sensor agreement within a zone;
- time since last valid observation;
- battery and communication health.

A stale or failed sensor must not be converted into zero. It should be represented through quality flags and missingness features.

1.6.4 5.4 Exposure features

Exposure does not determine whether a landslide occurs, but it determines operational priority. Include estimated population, buildings, schools, health centres, roads, bridges, critical infrastructure, shelters, and evacuation-route accessibility. Keep precise household and vulnerable-person data restricted.

1.7 6. Label construction

Do not train the model on an imprecise label such as “landslide happened during the monsoon.” Define the prediction horizon and target explicitly.

Recommended targets include:

Target	Definition
event_in_next_6h	Verified event intersects the zone within the next six hours.
event_in_next_24h	Verified event intersects the zone within the next 24 hours.
movement_anomaly	Quality-controlled sensor anomaly exceeds an approved threshold.
no_event_observed	Adequate observation and reporting coverage exists without a verified event.

Every event should retain its original source, date and time uncertainty, spatial uncertainty, verification status, event type, and label confidence. Unknown periods should remain unknown. Under-reporting can otherwise make an unsafe location appear safe.

1.8 7. XGBoost training pipeline

1.8.1 7.1 Extracting training data

```
import os
import pandas as pd
from sqlalchemy import create_engine

DATABASE_URL = os.environ["DATABASE_URL"]
engine = create_engine(DATABASE_URL)

query = """
SELECT *
```

```

FROM landslide_training_features
WHERE label_confidence >= 0.8
ORDER BY observation_time;
"""

df = pd.read_sql(query, engine)

feature_columns = [
    "slope_degrees",
    "elevation_m",
    "soil_moisture",
    "rainfall_1h_mm",
    "rainfall_6h_mm",
    "rainfall_24h_mm",
    "rainfall_72h_mm",
    "rainfall_forecast_6h_mm",
    "rainfall_forecast_24h_mm",
    "tilt_change_1h",
    "tilt_change_6h",
    "pore_pressure_change",
    "road_cut_distance_m",
    "drainage_distance_m",
    "historical_event_count",
    "exposed_population",
    "sensor_health_score"
]

X = df[feature_columns]
y = df["event_in_next_6h"].astype(int)

```

1.8.2 7.2 Training model

```

from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=400,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="binary:logistic",
    eval_metric="auc",

```

```

    tree_method="hist",
    random_state=42
)

model.fit(X_train, y_train)
risk_probability = model.predict_proba(X_test)[: , 1]

```

These values are starting points, not final production settings. Hyperparameters must be selected through validation and documented in a model card.

1.8.3 7.3 Converting output into risk states

```

def warning_state(probability, movement_detected=False):
    if movement_detected and probability >= 0.70:
        return "warning"
    if probability >= 0.60:
        return "watch"
    if probability >= 0.35:
        return "advisory"
    return "normal"

```

The thresholds above are examples only. They must be calibrated using local event prevalence, the cost of missed events, false-alarm tolerance, lead-time requirements, and authority approval.

1.9 8. Validation and model governance

1.9.1 8.1 Avoid data leakage

Do not randomly split adjacent observations from the same event across training and testing. That can place nearly identical measurements in both sets and produce an unrealistic score.

Use time-based validation, monsoon-season holdouts, leave-one-slope-out validation, and leave-one-region-out validation. The test set must represent future deployment conditions.

1.9.2 8.2 Metrics

Evaluate at the operational decision level. Required metrics include precision, recall, false-alarm rate, missed-event rate, lead time, calibration error, alert fatigue per community, and the percentage of alerts with complete evidence.

Report metrics separately by monsoon season, geology, district, sensor coverage, and warning state. Because landslides are rare, accuracy alone is not an acceptable primary metric.

1.9.3 8.3 Calibration and uncertainty

If the model output is presented as a probability, a value of 0.9 should be approximately correct 90 percent of the time within the same operating population. Otherwise, call it a risk score rather than a probability. Every prediction should include:

- risk state and score;
- model and feature version;
- feature timestamp;

- top evidence;
- missing or stale inputs;
- calibration or confidence status;
- recommended verification step;
- expiry time.

1.9.4 8.4 Explainability

Use feature importance or SHAP-based explanations for analysts. Example evidence may include “72-hour rainfall exceeded the local threshold,” “soil moisture is rising rapidly,” or “tilt acceleration is abnormal.” Resident messages should present the action, not a technical feature list.

1.10 9. Optional use of pgvector

If GeoRakshak later needs embeddings, PostgreSQL can store them with pgvector:

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE report_embeddings (
    report_id BIGINT PRIMARY KEY,
    embedding VECTOR(768),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

This is appropriate for similarity search and triage. It should not replace the structured XGBoost feature table.

The recommended architecture is:

```
PostgreSQL + PostGIS
|
+-- structured physical features
|   |
|   +-- XGBoost landslide-risk model
|
+-- optional pgvector embeddings
    |
    +-- image, text, and report similarity tools
```

1.11 10. Production architecture

The production pipeline should contain the following modules:

Module	Responsibility
Ingestion adapters	Retrieve weather, forecast, sensor, satellite, and community data.
Quality-control layer	Validate coordinates, timestamps, ranges, duplicates, clock drift, and sensor health.

Module	Responsibility
Feature builder	Generate zone-time feature snapshots with a feature-version identifier.
Risk engine	Run baseline rules and XGBoost inference.
Verification workflow	Route elevated signals to analysts or field responders.
Alert service	Issue approved messages via SMS, IVR, siren, public address, push, or dashboard.
Audit service	Record data, model, human decision, delivery, and acknowledgement history.
Monitoring	Track freshness, uptime, latency, failures, drift, and false alarms.

1.12 11. Security and privacy

Use role-based access control, strong operator authentication, encrypted transport, encrypted backups, secret management, signed sensor payloads, input validation, rate limits, dependency scanning, and append-only alert audit logs.

Separate public hazard maps from restricted contact lists, exact household coordinates, original media, and vulnerable-person information. Display approximate locations in public views. Define retention, deletion, consent, and access-logging policies before collecting personal data.

1.13 12. Phased implementation plan

1.13.1 Phase 1: baseline

Implement PostgreSQL/PostGIS, hazard zones, historical event schema, IMD ingestion, rainfall windows, transparent thresholds, and operator evidence display.

1.13.2 Phase 2: field data

Install a limited number of rain, soil-moisture, and tilt sensors. Implement edge buffering, sensor health, offline community reporting, local-language templates, and drill-only alerts.

1.13.3 Phase 3: XGBoost model

Train and calibrate XGBoost after sufficient local labels exist. Compare it against the baseline through time-based and spatial validation. Keep human approval for operational warnings.

1.13.4 Phase 4: optional embeddings

Add pgvector for report and image similarity, with human review. Treat computer vision and text models as triage tools rather than autonomous warning authorities.

1.14 13. Acceptance criteria

The implementation should be considered ready for a controlled pilot only when:

1. every feature has documented provenance and units;
2. every label has a verification and uncertainty policy;

3. model training can be reproduced from a versioned feature snapshot;
4. time-based and spatial validation has been completed;
5. stale and failed sensors reduce confidence instead of producing false safe values;
6. every assessment stores its model version, evidence, quality, and expiry;
7. high-consequence alerts require an authorized human decision;
8. alerts can reach people without smartphones through voice, SMS, siren, or local responders;
9. alert delivery, acknowledgement, and fallback channels are tested;
10. the system has rollback, audit, privacy, and incident-review procedures.

1.15 14. Conclusion

GeoRakshak should use PostgreSQL/PostGIS as the authoritative data layer and convert its structured data into versioned feature vectors for XGBoost. This is more appropriate than converting the entire database into generic embeddings because landslide risk depends on interpretable physical relationships among rainfall, terrain, soil, movement, and exposure.

The most reliable engineering path is to establish a transparent baseline first, build a high-quality local inventory, introduce XGBoost with strict spatial and temporal validation, expose evidence and uncertainty, and connect the result to a human-approved, multi-channel early-warning workflow.

1.16 References